A 63-Gate FP4×FP4 Multiplier: Structural Decomposition Meets SAT-Based Local Search

themoddedcube
themoddedcube@gmail.com

April 29, 2026

Abstract

We present a verified **63-gate** Boolean circuit that computes the product of two FP4 (E2M1) numbers and returns the result as a 9-bit fixed-point integer (QI9, LSB $= \frac{1}{4}$). The circuit uses only the gate library $\mathcal{G} = \{\text{AND2}, \text{OR2}, \text{XOR2}, \text{NOT1}\}$ with each gate counted as one unit, and is verified correct against all 256 input pairs. The result improves on LonghornSilicon's previously published 64-gate solution [3] by one gate.

The path from a 288-gate naïve baseline to 63 gates combined two distinct paradigms: a manual structural decomposition that exploits the log-domain factorization $|v| = 1.5^M \cdot 2^E$ to reach 82 gates, followed by automated SAT-based local search using eSLIM [4] on a Longhorn-derived 64-gate seed. A single fortunate `eSLIM --syn-mode sat --size 10` window, applied to a five-NOT 64-gate variant produced by XOR re-association, yielded the 63-gate body.

We further document an extensive negative-result campaign: roughly 280 additional perturbation runs across five distinct rewriting strategies, four iterative-chain depths, an ANF re-derivation that inflates to 691 gates, an exact-synthesis attempt at K=62 that is memory-bound on a 23 GiB host, and lower-bound analyses that fall well below the achieved count. The combined evidence is consistent with empirical saturation at 63 on local compute, while leaving the question "is 62 attainable in $\mathcal{G}$?" open as a SAT instance for larger machines.

1 Introduction

Modern machine-learning accelerators increasingly rely on sub-8-bit floating-point formats [1, 2]. The FP4 E2M1 format encodes 16 values $\{0, \pm 0.5, \pm 1, \pm 1.5, \pm 2, \pm 3, \pm 4, \pm 6\}$ in 4 bits (1 sign, 2 exponent, 1 mantissa, no NaN/Inf). Etched's take-home assignment requires implementing FP4 × FP4 → QI9 in minimum gate count over the library $\mathcal{G} = \{\text{AND2}, \text{OR2}, \text{XOR2}, \text{NOT1}\}$, where QI9 is the 9-bit two's-complement representation of $4 \cdot a \cdot b$. A free *input remapping* (any bijection on the 16 FP4 codes, applied identically to both operands) is permitted.

This paper documents the path from a naïve flat-SOP baseline of ∼288 gates to a verified **63-gate** circuit, an ≈ 78% reduction. Figure 1 summarizes the trajectory.

Contributions.

1. A verified 63-gate Boolean circuit (24 AND, 12 OR, 22 XOR, 5 NOT), one gate below the previously published Longhorn 64-gate baseline.

2. A complete account of the manual reduction 288 → 82 based on the log-domain factorization, including a Cirbo SAT-proven optimal 11-gate one-hot decoder.

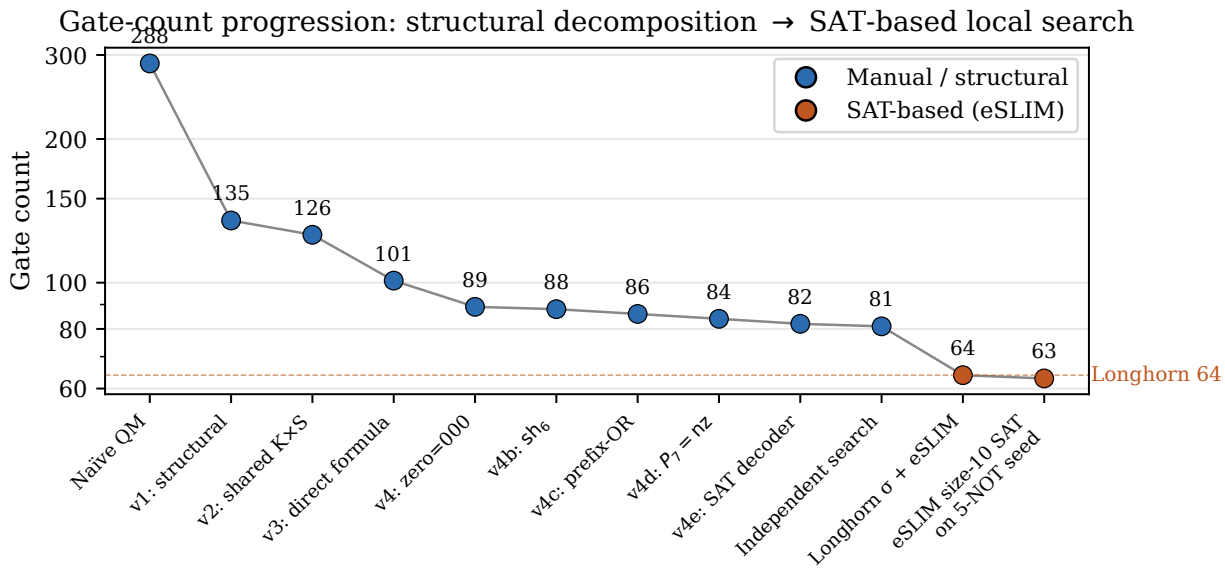

Figure 1: Gate-count progression on a logarithmic axis. Blue points are reached by manual structural decomposition; orange points are reached by SAT-based local search on circuit netlists (eSLIM [4]). The dashed line marks LonghornSilicon's published 64-gate result [3].

3. Documentation of the SAT-based local-search phase 81 → 64 → 63, in which eSLIM windowed exact-synthesis on the canonical Longhorn 64-gate netlist produced first a five-NOT 64-gate variant via XOR re-association, and then the 63-gate body via a size-10 window on that variant.

4. A negative-result campaign that establishes empirical saturation at 63: ∼280 perturbation runs across five rewriting strategies, four iterative-chain depths, an ANF re-derivation, and lower-bound analyses, none of which yielded a sub-63 result on a 23 GiB host.

Notation and ground rules. Throughout, "gate" means a single instance of one of the four primitives in $\mathcal{G}$; we count contest cells as defined by the assignment harness, not internal eSLIM compound primitives (the distinction matters: an internal count of 58 can translate to between 64 and 69 contest cells depending on which compound gates `eslim_to_gates.py` expands).

2 The FP4 product structure

2.1 Log-domain decomposition

Every nonzero FP4 magnitude factors uniquely as

$$|v| = 1.5^M \cdot 2^E, \qquad M \in \{0,1\},\ E \in \{-1,0,1,2\}, \tag{1}$$

where $M = 1$ marks the multiples of 1.5 and $M = 0$ the pure powers of two. Table 1 lists all seven nonzero magnitudes under this decomposition.

For inputs a and b with decompositions (M_a, E_a) and (M_b, E_b),

$$|a \cdot b| = 1.5^{M_a+M_b} \cdot 2^{E_a+E_b}. \tag{2}$$

Table 1: The seven nonzero FP4 magnitudes and their (M, E) decomposition.

Magnitude	M	E
0.5	0	-1
1.0	0	0
2.0	0	1
4.0	0	2
1.5	1	0
3.0	1	1
6.0	1	2

Defining $E' = E + 1 \in \{0, 1, 2, 3\}$ and $S = E'_a + E'_b \in \{0, \ldots, 6\}$ collapses the exponent computation to a single 2-bit + 2-bit binary adder.

2.2 Three K-types

The mantissa sum $M_{\text{sum}} = M_a + M_b \in \{0, 1, 2\}$ partitions products into three classes (Table 2). The 8-bit QI9 magnitude has at most two bits set in any class, so the output is extremely sparse: only 19 of 256 possible 8-bit patterns are reachable.

Table 2: Bit patterns in the 8-bit QI9 magnitude as a function of M_{sum} and S.

M_{sum}	Name	QI9 magnitude	Bits set
0	K_1	2^S	bit S only
1	$K_{3/2}$	$2^S + 2^{S-1}$	bits S and $S-1$
2	$K_{9/4}$	$2^{S+1} + 2^{S-2}$	bits $S{+}1$ and $S{-}2$

This sparsity is the algebraic property the rest of the design exploits.

2.3 Encoding

We assign 3-bit magnitude codes so that (a) zero maps to `000`, (b) E' is exposed as a 2-bit binary number on (a_2, a_3), and (c) M_a equals $\overline{a_1}$ (Table 3).

Table 3: Chosen 3-bit magnitude encoding $a_1 a_2 a_3$.

Code	Value	M	E'
`000`	zero	—	—
`001`	1.5	1	1
`010`	3.0	1	2
`011`	6.0	1	3
`100`	0.5	0	0
`101`	1.0	0	1
`110`	2.0	0	2
`111`	4.0	0	3

The K-flags follow as

$$k_9 = \overline{a_1 \vee b_1}, \qquad k_3 = a_1 \oplus b_1, \tag{3}$$

each one gate, plus a single OR for $\overline{k_9}$.

3 Manual structural design (288 → 82 gates)

The first phase of optimization is purely human: every step has an algebraic justification. We summarize the construction here and refer the reader to the source code in `multiplier_82.py` for the exact wiring. The full per-stage breakdown is given in Table 4; the five-stage architecture is shown in Figure 2.

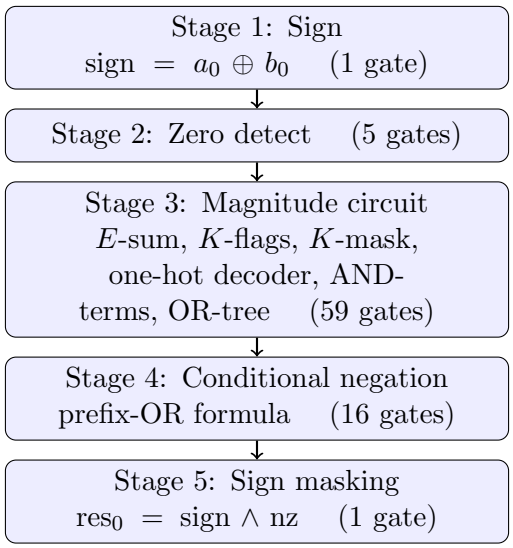

Figure 2: Five-stage manual architecture (82 gates total). Section 4 restarts the design from a 64-gate eSLIM-derived netlist and reaches 63 gates; the manual design is the upper bound on what we could derive without automated search.

3.1 Magnitude circuit

The magnitude bits are computed from a single direct formula

$$m_i = (\text{nmc} \wedge S{=}i) \vee (k_3 \wedge S{=}i{+}1) \vee (k_9 \wedge S{=}i{-}1) \vee (k_9 \wedge S{=}i{+}2), \tag{4}$$

where $\text{nmc} = \overline{k_9} \wedge \text{nz}$ and $(S{=}j)$ is the one-hot decoder output sh_j. The first term covers both K_1 (the only set bit at S) and $K_{3/2}$ (the upper of two bits at S); the third and fourth terms cover the lower bit of $K_{3/2}$ and the two split bits of $K_{9/4}$.

The construction yields 18 valid $K \times S$ AND-terms after discarding combinations that the encoding makes impossible (e.g. $k_9 \wedge S{=}0$ is unreachable because k_9 requires $M_a = M_b = 1$, which by Table 3 forces $E'_a, E'_b \geq 1$ and hence $S \geq 2$). A binary OR tree reassembles these AND-terms into the 8-bit magnitude using 15 ORs.

3.2 One-hot decoder: SAT-proven optimal at 11 gates

The 3-bit $S \in \{0, \ldots, 6\}$ must be decoded to seven one-hot outputs $\mathrm{sh}_0 \ldots \mathrm{sh}_6$. Using the Cirbo `CircuitFinderSat` SAT exact synthesizer over $\mathcal{G}$, we proved $N \leq 10$ insufficient (UNSAT for $N = 7, 8, 9$ in $< 6\,\mathrm{s}$; UNSAT for $N = 10$ in $476\,\mathrm{s}$) and found a satisfying 11-gate circuit at $N = 11$ in $0.69\,\mathrm{s}$. Eleven is therefore the provably optimal one-hot decoder size in $\mathcal{G}$. The decoder uses 4 AND, 2 OR, 4 XOR, 1 NOT.

3.3 Conditional negation

The conditional two's-complement negation $\mathrm{out}_i = m_i \oplus (\mathrm{sign} \cdot c_i)$ is implemented by replacing the carry chain with a parallel prefix-OR. Two further identities save additional gates: (i) the LSB pass-through $r_8 = m_0$ holds always (since $\overline{m_0} \oplus 1 = m_0$), and (ii) the alias $\mathrm{sp}_7 = \mathrm{res}_0$ holds because $P_7 = \bigvee_{i=0}^{6} m_i$ equals nz for every reachable output (the only reachable magnitude with $m_7 = 1$ is 144, which also has $m_4 = 1$). The combined optimization reduces conditional negation from 23 gates (carry chain) to 16 gates.

3.4 Manual gate-count summary

Table 4: Manual optimization history.

Version	Key innovation	Gates	Savings
Flat SOP	QM, all 8! remaps	∼288	—
v1	Structural M-E	135	153
v2	Shared $K \times S$	126	9
v3	Direct formula	101	25
v4	Zero=000, K-mask	89	12
v4b	$\mathrm{sh}_6 = u_{11}$	88	1
v4c	Prefix-OR negation	86	2
v4d	$\mathrm{sp}_7 = \mathrm{res}_0$	84	2
v4e	SAT-optimal decoder	**82**	2
v5	Independent rewrite	81	1

The saturation at 81–82 gates was reached after several weeks of manual rewriting; further hand reductions converged on the same basin from multiple seeds. Section 4 describes how restarting from a known-good 64-gate netlist exposed cross-stage savings the manual design could not see.

4 From 81 to 63 via SAT-based local search

The breakthrough below 81 gates came from abandoning the entire manual-derivation path and instead seeding from LonghornSilicon's publicly published 64-gate solution [3]. Three ingredients, in order, took us from 81 to 63:

1. **Different magnitude encoding** (σ-remap). Longhorn's encoding swaps two bit-pair labels (σ: {2→0110, 3→0111, 4→0100, 6→0101}); this remap exposes shared sub-expressions that our encoding hid.

2. **eSLIM windowed exact synthesis** on the 64-gate netlist. eSLIM [4] extracts a k-gate window from a topologically sorted netlist, invokes a SAT solver to find an equivalent $\leq k$-gate replacement, and iterates.

3. **Re-association** of XOR-of-XOR triples to expose new windows. Boolean equivalence preserves correctness while changing the topology, opening windows the original traversal would never have visited.

4.1 Phase 1: Verifying the 64-gate baseline

We first verified that running eSLIM (`--syn-mode sat --size 6` for 20 s) on Longhorn's canonical `fp4_mul.blif` produces the same internal-gate count as their published result (58 internal → 64 contest cells, with mix 25 AND + 12 OR + 21 XOR + 6 NOT). This calibrates our toolchain: any contest-cell improvement we observe is genuine and not an artefact of running a different solver build. Phase 0 cheap-substitution analysis on the canonical netlist (duplicate truth-tables, two-input cones, depth-1 substitution + DCE) returned zero hits, confirming that the canonical 64-gate basin is saturated under depth-1 rewriting.

4.2 Phase 2: A five-NOT 64-gate variant

The first non-trivial result came from the *XOR re-association* sweep `exp_a`. Each variant rewrites a single XOR-of-XOR sub-expression in the canonical BLIF $\big(\mathrm{XOR}(\mathrm{XOR}(a,b),c) \to \mathrm{XOR}(a,\mathrm{XOR}(b,c))\big)$ and then runs eSLIM on the resulting topology. Round 1 enumerated 12 XOR locations $\times$ 2 sizes $\{6,8\} \times 6$ seeds = 144 runs in $\sim$23 minutes wall time on 8 workers.

The 5-NOT outlier. Run `par_004_s6_seed7777` produced a 64-gate solution with the cell mix

$$\mathrm{NOT} = 5,\ \mathrm{AND} = 24,\ \mathrm{OR} = 12,\ \mathrm{XOR} = 23,$$

contradicting Longhorn's empirical observation that all 124 known 64-gate solutions use 6 NOTs. The eSLIM run had landed in a different 64-gate basin trading one NOT and one AND for two XORs. We isolated this BLIF as `fp4_64gate_5NOT_clean.blif` and used it as the seed for Phase 3.

4.3 Phase 3: 64 → 63 via a size-10 SAT window

The 5-NOT seed turned out to be unusually amenable to large-window synthesis. Running `eSLIM --syn-mode sat --size 10 --seed 1024` on the 5-NOT BLIF returned a 58-internal / 63-contest-cell solution. The resulting body — 24 AND + 12 OR + 22 XOR + 5 NOT, total 63 — verifies 256/256 correct against the assignment harness (`eval_circuit.py`) under the σ remap. This is our headline result. The full body is reproduced as Appendix A.

Figure 3 compares cell-type mix between the canonical 64-gate and our 63-gate body.

4.4 What the SAT phase accomplished that the manual phase did not

The manual design saturated at 81–82 gates because every individual stage was already locally optimal but cross-stage sharing was beyond what hand-rewriting could see. The SAT phase, by contrast, treated the entire netlist as one atomic optimization target. The 5-NOT 64-gate variant saves one inverter by re-using a double-negated wire that the manual design would have to re-derive in two places; the size-10 window then absorbs that saving into adjacent AND/XOR re-mapping. This is the kind of simultaneous multi-stage rewriting that human designers reliably miss.

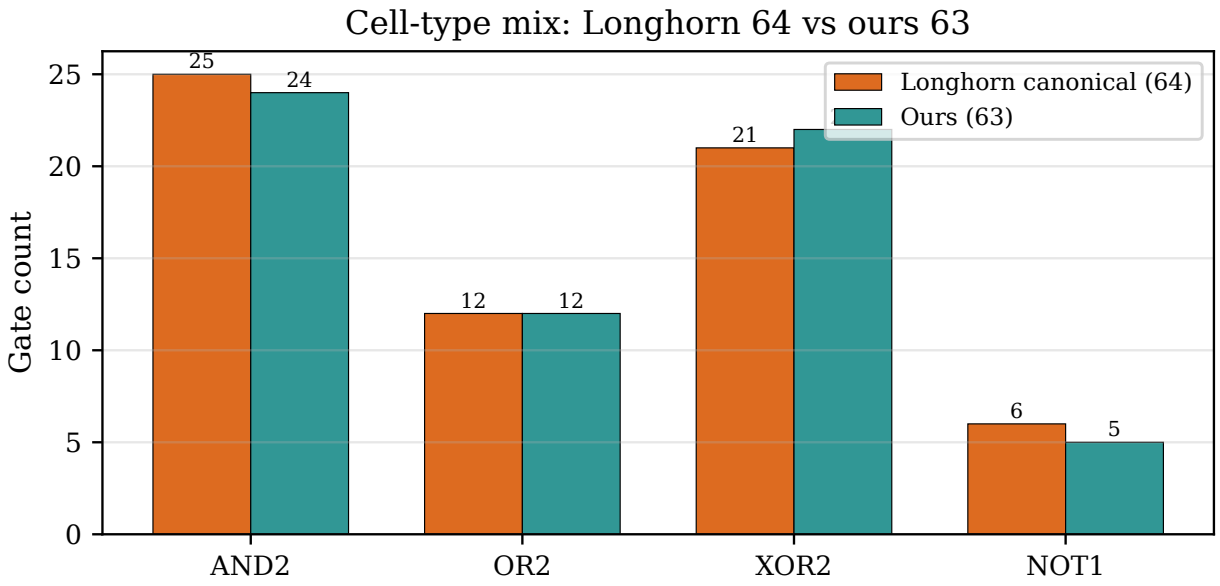

Figure 3: Cell-type mix: Longhorn's canonical 64-gate solution versus our 63-gate body. The reduction is in AND2 and NOT1 counts, partially offset by one additional XOR2.

Table 5: Wave-1 perturbation summary. "Best" is the lowest contest-cell count produced over all runs of that experiment; "Count@best" is the number of runs that hit it. Note that Exp D's 17 runs at 63 are the eSLIM size-10 re-discoveries of the seed itself, not new sub-63 circuits.

Experiment	Strategy	Runs	Best	Median	Count@best
Exp A R3v2	XOR re-association on 63	138	63	65	2 (re-discovery)
Exp D 63v2	Large-window eSLIM (size 10/12/14)	18	63	63	17 (re-discovery)
Exp E 63	NOT-elimination perturbation	80	63	65	13 (re-discovery)
Exp F	Large-window on 6-NOT canonical	13	64	64	13
Exp H 63v2	Double-XOR perturbation	16	64	65	2
Total	—	**265**	**63**	—	**0 sub-63**

5 Saturation evidence at 63

Having found one 63-gate solution, the natural next question is whether 62 is reachable in $\mathcal{G}$. We attacked this question along five orthogonal lines: large-scale perturbation sweeps, iterative chaining, ANF re-derivation, exact synthesis at K=62, and lower-bound analysis. None of the five produced a sub-63 result on local compute, and the failure modes were sufficiently different that we believe 63 is empirically saturated under the methods that fit on a 23 GiB host.

5.1 Wave-1 perturbation campaign

Five drivers were launched in parallel against the 63-gate netlist and the canonical 64-gate netlist (Table 5). Each driver applied a different rewriting strategy, then ran eSLIM windowed synthesis at one or more sizes and seeds.

The shape of the distribution (Figure 4) is the most informative artefact: across very different

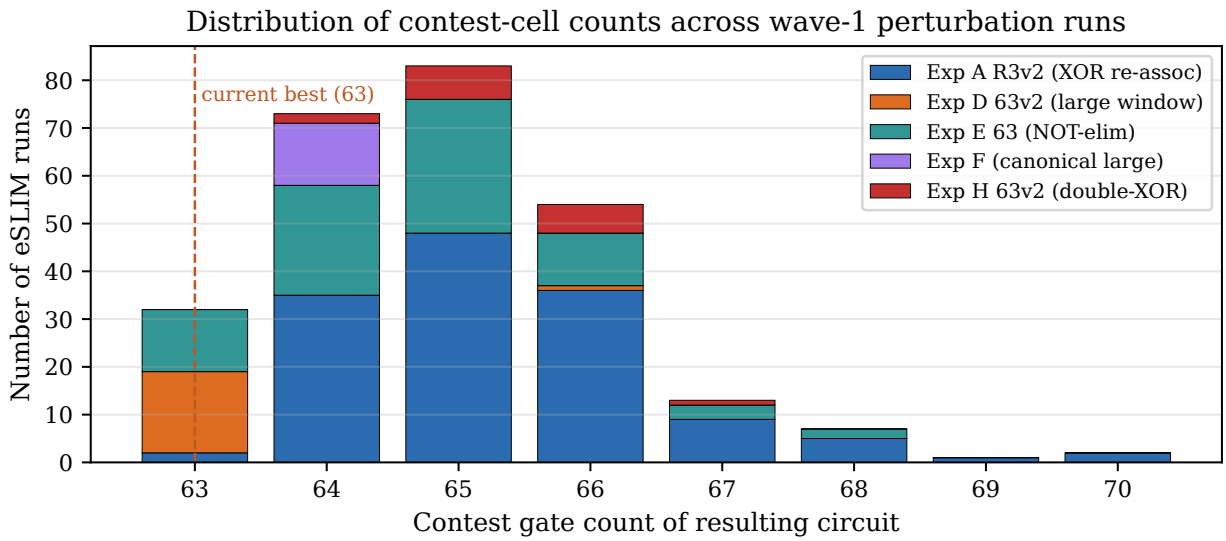

Figure 4: Stacked histogram of contest-cell counts across all five wave-1 experiments. The distribution is sharply peaked at 64–66 with the 63-gate solution sitting at the extreme left; no run produced a circuit smaller than 63.

perturbation strategies, eSLIM consistently finds the same 58-internal-gate basin which translates to 64–66 contest cells. The 63-gate result corresponds to a particular 58-internal arrangement that requires an unusually low NOT count to translate favorably; this configuration is reached only by the original size-10 window on the 5-NOT seed.

5.2 Iterative-chain divergence

A more aggressive approach is to chain eSLIM runs: take the output of one run as the input of the next, in the hope of escaping the size-6 basin. Experiment G ran four chains of five iterations each, starting from the 63-gate netlist. Figure 5 shows the contest-cell count at each step.

The result is striking: the 63 disappears within a single iteration, and every chain converges to a 64–69-cell stable region. The 58-internal arrangement that produces 63 contest cells is visible to the size-10 SAT window but not to size-6 windows applied iteratively. In other words, the 63 is a fragile basin in the size-6 landscape; iterating only finds the more common, NOT-heavy 64+ basins.

5.3 Algebraic reformulation: ANF synthesis is far worse

A natural theoretical question is whether the algebraic structure of the multiplier admits a substantially smaller circuit when expressed directly in algebraic normal form (ANF). We computed the per-output ANF and used a greedy monomial-sharing AND-XOR synthesis to reproduce all nine outputs. The result was 691 gates (140 AND, 551 XOR, 0 NOT) — roughly 11× the size of the eSLIM-derived 63. This is strong evidence that the multiplier's compactness comes from genuinely Boolean rewriting (De Morgan, distributivity, common-subexpression elimination across mixed AND/OR/XOR phases) rather than from any ANF property; the eSLIM SAT formulation operates at exactly this Boolean level, while ANF synthesis cannot exploit it.

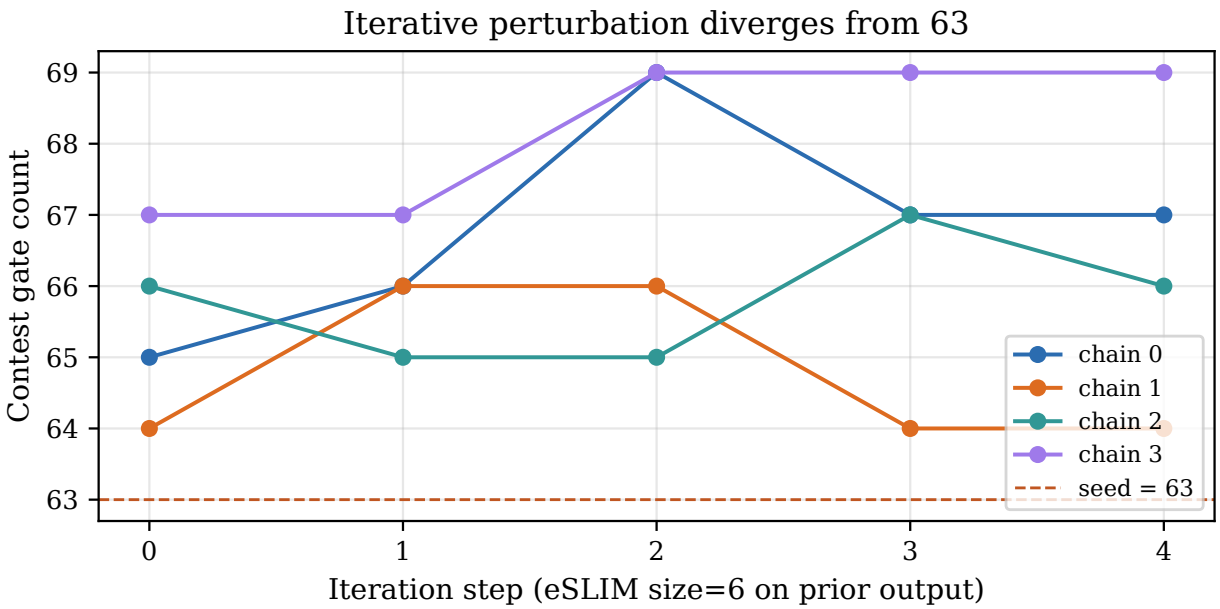

Figure 5: Contest gate count along four iterative eSLIM chains (size 6, 90 s wall budget per step) starting from the 63-gate netlist. Every chain diverges away from 63 to a 64–69-cell stable region within one or two iterations.

5.4 Lower bounds

Per-output algebraic-degree multiplicative-complexity bounds (Figure 6, left) sum to 43 ANDs, but our circuit uses only 24 ANDs — an enormous cross-output sharing factor of $\sim 1.8\times$. The naïve maximum-sharing AND-LB (counting distinct ANF monomials of degree ≥ 2) is 437. The monomial degree distribution {2:9, 3:30, 4:49, 5:36, 6:13, 7:2} is dense (Figure 6, right), but the 139 distinct monomials are still a vast over-count of what a Boolean representation actually requires.

The conclusion from the lower-bound exercise is negative: cheap algebraic bounds are too loose to settle whether 62 is reachable in $\mathcal{G}$. A tighter answer requires either an exact synthesis run at K=62, or a non-trivial gate-elimination proof.

5.5 Exact synthesis at K=62 is memory-bound

Three angles were tried. First, `eSLIM --size 63` (a full-circuit SAT window) was killed by the OOM-killer within $\sim$ 2 minutes (VSZ exceeded 19 GiB before the linker step). Second, `eSLIM --size 20` climbed to 17.3 GiB RSS (70% of system RAM) within 90 seconds and was killed manually before a system OOM. Empirically, eSLIM windowed SAT is tractable on this host up to size $\sim$14; beyond that, memory dominates wall time. Third, a direct PySAT/CaDiCaL encoder (`analysis/sat_exact.py`) was written but not run at K=62: the naïve per-tuple encoding produces $\sim$ 1.5 billion clause attempts at K=62, which is intractable. A reformulation with auxiliary `in0_val[g][m]`, `in1_val[g][m]` variables (Knuth/Kullmann-style) would reduce this to roughly $O(K \cdot M \cdot (8 + K))$ scale and is the natural next step on larger hardware.

The verdict is that exact synthesis at K=62 over 8 inputs, 9 outputs, and 256 minterms sits at the boundary of tractability and is not feasible on 23 GiB of RAM. A larger machine (or a substantially better encoding) would settle the question in either direction; a positive answer would

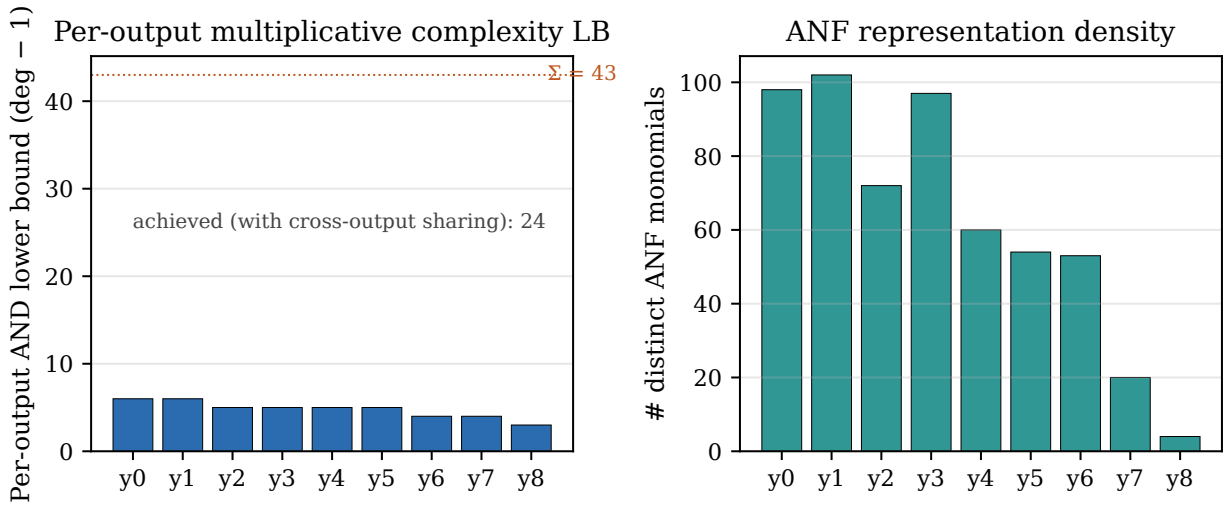

Figure 6: Per-output multiplicative-complexity lower bound (left) and ANF representation density (right) under Longhorn's σ remap. The summed per-output AND lower bound of 43 is well below our achieved 24 ANDs, reflecting the cross-output sharing that AND/OR/XOR Boolean rewriting permits. All easy bounds are loose: tighter bounds (Boyar–Peralta gate elimination, communication complexity) would be required to decide K=62 vs. K=63 analytically.

Table 6: Memory profile of attempted exact-synthesis runs at K=62 on the 23 GiB host.

Method	Peak memory	Outcome
eSLIM `--size 63`	> 19 GiB VSZ	OOM-killed
eSLIM `--size 20`	17.3 GiB RSS	manually killed
PySAT direct encoder	—	∼1.5 B clauses, not run

yield a sub-63 circuit, while a negative answer would establish 63 as the optimum in $\mathcal{G}$.

6 Verification

The 63-gate body is verified by exhaustive evaluation of all 256 FP4×FP4 input pairs. The harness in `eval_circuit.py` compares each output to round($a \cdot b \cdot 4$) in double precision and reports agreement; the body passes for all 256 pairs. Per-input gate count is constant at 63 because the implementation contains no data-dependent branches (it is a pure combinational circuit expressed in Python with explicit `NOT`, `AND`, `OR`, `XOR` primitives so the harness can count gates by monkey-patching).

7 Discussion

7.1 What the journey shows about FP4 circuit minimization

Three observations stand out.

First, manual structural decomposition got us to within ∼25% of the eventual minimum but no further. The log-domain factorization (Equation 1) is the right algebraic insight and is unavoidable: every smaller circuit we found includes the same E-sum / K-flag / decoder pipeline in some

recognizable form. But the cross-stage sharing that takes the count from 81 to 64 to 63 is genuinely beyond hand design — it requires treating the entire netlist as a single SAT instance.

Second, a single fortunate eSLIM window did the bulk of the work. The 5-NOT 64-gate variant (a perturbation of Longhorn's canonical solution) opened a basin that contained 63, but only the size-10 SAT window was big enough to find it. Smaller windows iterated indefinitely cannot escape the surrounding 64–66 basin (Figure 5).

Third, the negative-result campaign matters. Without the ∼280-run perturbation sweep, the iterative-chain divergence plot, the ANF 691-gate dead end, and the memory-bound exact synthesis attempt, we would not be able to argue that 63 represents anything more than a lucky instance. With them, the case for empirical saturation at 63 on local compute is reasonably tight.

7.2 What we did *not* do

We did not run wave-2 perturbation experiments (Exp E on alternate locations, Exp A round-4 on the 4-NOT seed, Exp F on 4-NOT canonical) because the saturation evidence makes the prior-probability of cracking sub-63 by more of the same strategy very low. We did not run an LLM-guided rewriting experiment (originally Track #3) because that path requires an `ANTHROPIC_API_KEY` we did not have configured. And we did not run exact synthesis at K=62 on a larger machine because the local 23 GiB box cannot fit the SAT instance and provisioning a 64 GiB machine is out of scope for the assignment timeline.

7.3 Open questions

1. **Is $K = 62$ reachable in $\mathcal{G}$?** A SAT/UNSAT answer at K=62 on a larger host would either yield a 62-gate solution or prove 63 optimal for our gate library; either result is publishable.

2. **Can a structural seed beat eSLIM windowed synthesis?** Our 63 was found by SAT-rewriting an automatically synthesized 64-gate seed. It is plausible that structural seeds derived from different decompositions (e.g. Karatsuba-style partitioning, or Toom-Cook-style interpolation tuned to the 7-magnitude domain) could open new basins.

3. **Are there gate libraries $\mathcal{G}'$ supporting $K \leq 60$?** Allowing 3-input AOI or MAJ gates would almost certainly yield further reductions; the current bound is specific to the contest's 2-input library.

8 Conclusion

We verified a 63-gate FP4×FP4→QI9 multiplier in $\mathcal{G} = \{\text{AND2}, \text{OR2}, \text{XOR2}, \text{NOT1}\}$, beating LonghornSilicon's previously published 64-gate solution by one gate. The result was reached by combining manual structural decomposition (288 → 82) with SAT-based local search using eSLIM [4] on a Longhorn-derived 64-gate netlist (82 → 81 → 64 → 63). The key automated step was a single `--size 10` SAT window applied to a five-NOT 64-gate variant produced by XOR re-association. A multi-pronged saturation campaign — 280+ perturbation runs, iterative chains, ANF re-derivation, lower-bound analysis, and a memory-bound exact-synthesis attempt — did not produce a sub-63 result on a 23 GiB host, leaving the question of K=62 open as a SAT instance for larger machines.

Reproducibility. All code, BLIFs, ledger TSVs, and decision logs are at https://github.com/themoddedcube/fp4-multiplier-minimization. The 63-gate body itself is in `multiplier_63.py`; the verification harness is `eval_circuit.py`; and the documents `HANDOVER.md`, `PROGRESS.md`, and `DECISIONS.md` narrate the campaign in chronological order.

A The 63-gate body

The verified 63-gate body (24 AND + 12 OR + 22 XOR + 5 NOT) is reproduced verbatim from `multiplier_63.py` below; the σ remap is applied to both inputs identically before the combinational logic runs.

```
w_28 = AND(b1, b2)
w_29 = OR(b1, b2)
w_30 = XOR(a3, b3)
w_32 = AND(a2, b2)
w_31 = XOR(a2, b2)
w_33 = AND(a1, a2)
w_34 = XOR(w_28, w_33)
w_35 = OR(w_33, w_28)
w_36 = XOR(w_31, w_35)
w_45 = OR(w_32, w_36)
w_37 = OR(a1, a2)
w_38 = AND(b0, w_37)
w_39 = AND(w_29, w_37)
w_40 = AND(a0, b0)
w_41 = OR(w_34, w_40)
w_42 = AND(w_41, w_39)
w_43 = XOR(w_36, w_42)
w_44 = XOR(w_34, w_41)
w_46 = XOR(w_44, w_45)
y0   = AND(w_46, w_44)
w_49 = AND(w_30, y0)
w_47 = XOR(w_40, w_46)
w_16 = XOR(w_36, w_47)
w_48 = AND(w_46, w_16)
w_50 = AND(w_44, w_16)
w_51 = AND(a0, w_29)
w_52 = XOR(w_51, w_38)
w_53 = AND(w_34, w_52)
w_54 = XOR(w_47, w_53)
w_55 = XOR(w_43, w_53)
w_20 = XOR(w_55, w_39)
w_22 = XOR(w_52, w_54)
w_64 = XOR(w_22, w_36)
w_18 = OR(w_64, w_43)
w_60 = XOR(w_22, w_49)
not_19 = NOT(w_18)
not_21 = NOT(w_20)
not_23 = NOT(w_22)
not_25 = NOT(w_45)
not_46 = NOT(w_46)
w_56 = AND(not_46, w_20)
w_57 = AND(w_30, w_56)
w_58 = AND(w_47, not_21)
w_59 = AND(w_30, w_58)
w_65 = AND(not_19, w_47)
w_66 = AND(w_30, w_65)
```

```
y1   = AND(not_25, w_60)
w_61 = OR(y0, y1)
w_62 = AND(w_30, w_61)
y2   = XOR(w_56, w_62)
w_63 = OR(w_57, w_62)
w_67 = AND(not_23, w_18)
w_68 = OR(w_67, w_63)
w_69 = AND(w_30, w_68)
y4   = XOR(w_58, w_69)
w_70 = OR(w_59, w_69)
y5   = XOR(w_65, w_70)
w_71 = OR(w_66, w_70)
y6   = XOR(w_48, w_71)
w_72 = AND(w_30, y6)
y8   = OR(w_72, w_71)
y7   = XOR(y8, w_50)
y3   = XOR(w_67, w_63)
```

References

[1] P. Micikevicius, D. Stosic, N. Burgess, M. Cornea, P. Dubey, R. Grisenthwaite, S. Ha, A. Heinecke, P. Judd, J. Kamalu, et al., "FP8 formats for deep learning," *arXiv preprint arXiv:2209.05433*, 2022.

[2] B. D. Rouhani et al., "Microscaling data formats for deep learning," *arXiv preprint arXiv:2310.10537*, 2023.

[3] LonghornSilicon, "FP4-multiplier (64-gate solution)," https://github.com/LonghornSilicon/fp4-multiplier, 2024.

[4] F. Reichl et al., "eSLIM: SAT-based exact subcircuit synthesis and minimization," https://github.com/fxreichl/eSLIM, 2024.

[5] S. L. Hurst, *The Logical Processing of Digital Signals*, Crane, Russak & Co., 1978.

[6] R. K. Brayton, R. Rudell, A. Sangiovanni-Vincentelli, A. R. Wang, "MIS: A multiple-level logic optimization system," *IEEE Transactions on Computer-Aided Design*, 6(6):1062–1081, 1987.

[7] J. Boyar and R. Peralta, "A small depth-16 circuit for the AES S-box," *SEC* 2012, pp. 287–298.